



NEXUS:

Advanced Static Analysis & Threat Intelligence

Senior Design II Final Presentation | COMP 4208 P02

Tuesday, April 28, 2026

Special Thanks To:



**Course
Instructor:**
Dr. Noushin
Ghaffari



**Snowflake
Senior Engineer:**
Jonathan
Martindale



**NSA Academic
Liaison:**
Dr. Gregory
Stevenson



**NSA Software
Engineer:**
Mr. Andrew
Hutton

Meet Our Team



Karrington Hall
Project Lead

Bachelor of Science in
Computer Science
Cybersecurity Concentration



Brandon Nobles
Backend Developer

Bachelor of Science in
Computer Science
Cybersecurity Concentration



Kendall Brown
UI/UX Developer

Bachelor of Science in
Computer Science
Data Science Concentration



LeMikkos Starks
RegEx & Data Evaluator

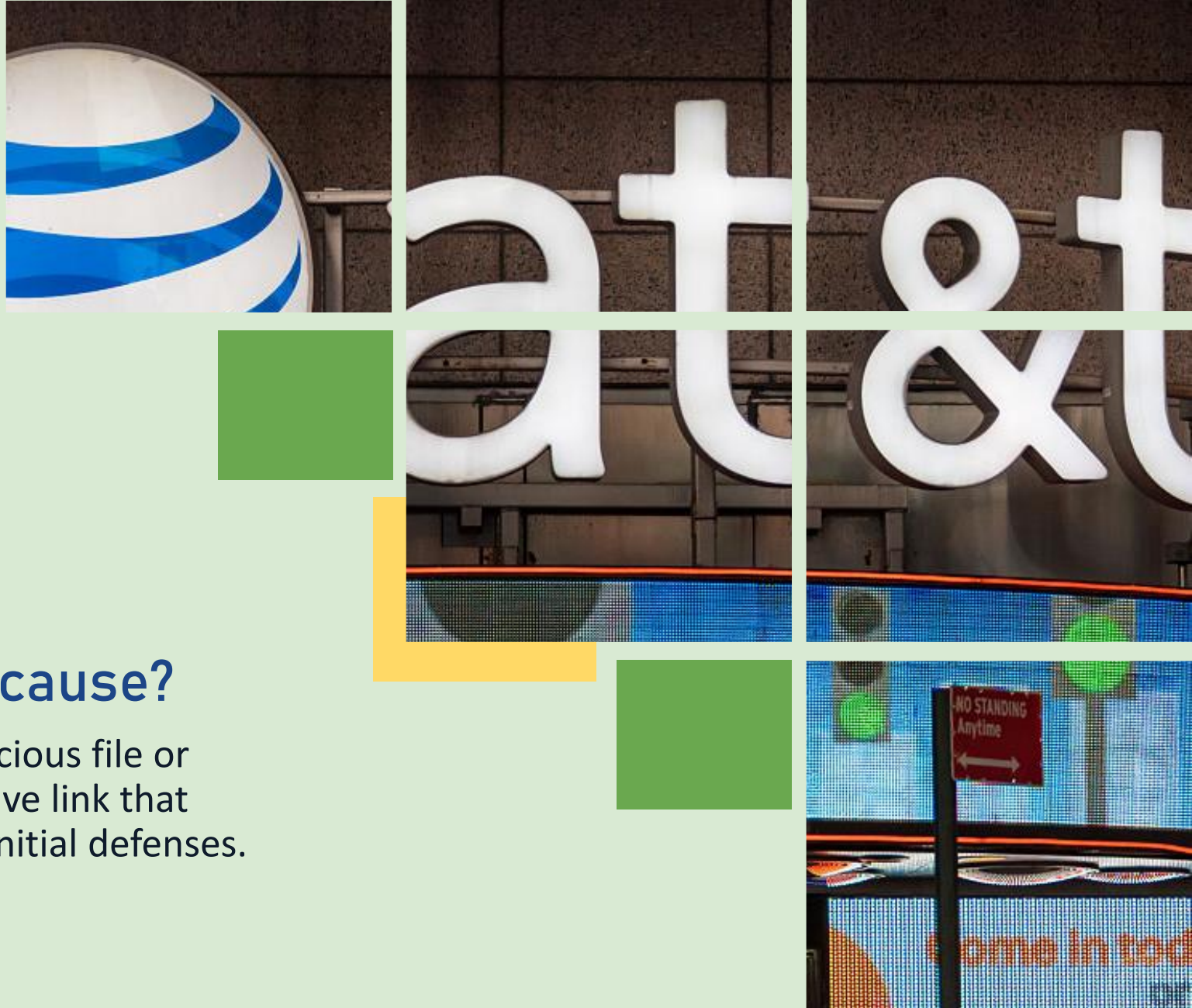
Bachelor of Science in
Computer Science
Data Science Concentration

Why Threat Analysis Matters: AT&T Data Breach – March 2024

- 73 million customer records exposed
- Cost: \$13 million in direct losses
- Reputation damage: immeasurable

The cause?

A suspicious file or deceptive link that bypassed initial defenses.



AGENDA

1. Project Genesis & Tech Stack
2. Backend Implementation
3. Scoring Engine & Pattern Detection
4. Frontend Implementation
5. Testing & Validation Results
6. Live Demonstration
7. Conclusion



1. PROBLEM STATEMENT



The Cybersecurity Gap

By The Numbers:

\$4.45M

Average cost of a data breach
(IBM, 2023).

45%

Percentage of malware
delivered through files, links,
and phishing content.

92%

Percentage of organizations that
rely on signature-based
detection.

2-4 hrs

Manual analysis of a suspicious
file can take hours depending
on complexity.

The Challenge:



Fast automated tools may miss
sophisticated or novel threats.



Thorough analysis takes too long



Existing solutions lack transparency



Privacy concerns with third-party
uploads

Existing Tools: Limitations



-
- ✓ Fast signature-based scanning
 - ✗ Limited explainability
 - ✗ May miss new or obfuscated threats
 - ✗ Limited metadata/context analysis



-
- ✓ Useful quick triage
 - ✗ Limited control / privacy
 - ✗ third-party file submission concerns
 - ✗ external dependency

The Gap:

Many existing tools require tradeoffs between speed, transparency, privacy, or deployment control.

2. OUR SOLUTION: NEXUS



	Results	Privacy	Speed	Methods
Other Tools	Often limited explanation	May require external services	Often tradeoffs in speed or depth	Limited workflow flexibility
NEXUS	Why it's malicious (ips, urls, metadata)	Local processing - Files never stored	Fast and accurate (<8 sec, 90% accuracy)	File, URL, email (3 analysis methods)

Our Distinctive Approach



3. PROJECT TIMELINE & STRUCTURE

From Concept to Capstone



Fall 2025

Jan 2026

Feb 2026

Mar 2026

Apr 2026

Discovery

- ✓ NSA project brief received
- ✓ Team Opulence formed
- ✓ Requirements gathered

Architecture

- ✓ Flask + Snowflake architecture designed
- ✓ RegEx pattern library drafted
- ✓ API contract defined

Backend MVP

- ✓ All 3 API endpoints built
- ✓ OCR pipeline integrated
- ✓ File processor handles 8 formats

Integration

- ✓ React frontend connected
- ✓ Dark/light theming added
- ✓ Results dashboard live

Validation

- ✓ 1,000-file test suite run
- ✓ Scoring calibrated
- ✓ IAC presentation delivered



REPOSITORY & FILE STRUCTURE

- 6 backend Python modules
- 4 frontend page components
 - 3 API endpoints
 - 15 RegEx patterns
- ~ 2,500 lines of backend code
- ~ 3,000 lines of frontend code

```
malicious-file-analyzer/
├── backend/
│   ├── services/
│   │   ├── file_processor.py
│   │   ├── scoring.py
│   │   ├── snowflake_client.py
│   │   ├── real_snowflake_client.py
│   │   ├── mock_snowflake_client.py
│   │   └── url_downloader.py
│   ├── utils/
│   │   └── regex_patterns.py
│   ├── test_data/
│   │   ├── malicious/
│   │   └── clean/
│   ├── tests/
│   ├── app.py
│   ├── requirements.txt
│   └── .env.example
├── frontend/frontend-app/
│   ├── public/
│   │   └── OpulenceLogo.png
│   ├── src/
│   │   ├── components/
│   │   │   ├── Nav.jsx
│   │   │   └── Footer.jsx
│   │   ├── pages/
│   │   │   ├── Home.jsx
│   │   │   ├── URLAnalyzer.jsx
│   │   │   ├── FileUpload.jsx
│   │   │   └── Results.jsx
│   │   ├── App.jsx
│   │   ├── App.css
│   │   └── index.js
│   ├── package.json
│   └── tailwind.config.js
├── docs/
│   ├── wireframes/
│   ├── API_REFERENCE.md
│   ├── AWS_CONFIGURATION.md
│   ├── BACKEND_IMPROVEMENTS.md
│   ├── EMAIL_ARCHITECTURE.md
│   ├── SNOWFLAKE_SETUP_GUIDE.md
│   ├── TEAM_ONBOARDING.md
│   └── UI_DESIGN.md
├── scripts/
│   └── setup_snowflake.sql
├── .gitignore
├── README.md
└── LICENSE
```

Python Flask API
Business logic
✓ Multi-format file parsing
✓ Threat scoring algorithm
✓ Snowflake factory pattern
✓ Production Snowflake client
✓ Development mock client
✓ Safe URL fetching

Helper functions
✓ 15+ indicator patterns

Test datasets
✓ 200+ malicious samples
✓ 200+ clean samples

Unit & integration tests
✓ Main Flask application
✓ Python dependencies
✓ Environment variables

React application
Static assets
✓ Team logo

UI components
✓ Navigation bar
✓ Footer component

Page components
✓ Landing page
✓ URL analysis
✓ File upload (.eml support)
✓ Results display

✓ Main component
✓ Global styles
✓ Entry point

✓ Dependencies
✓ Tailwind configuration

Documentation
✓ UI design mockups
✓ REST API documentation
✓ AWS setup guide
✓ Code review notes
✓ Email processing design
✓ Database setup
✓ Dev environment setup
✓ Frontend specifications

Utility scripts
✓ Database schema

✓ Git ignore rules
✓ This file
MIT License



4. SYSTEM ARCHITECTURE



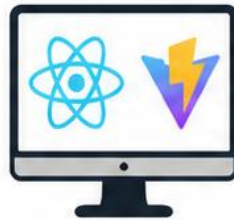
Technology Stack



BACKEND

Flask (Python 3.11)

API routing, request handling



FRONTEND

React.js + Vite

User Interface, SPA



DATABASE

Snowflake

Scan history, 30-day retention



OCR ENGINE

Tesseract

Extract hidden text from images



CLOUD SERVICES

AWS (S3/Lambda/SES)

Future email automation

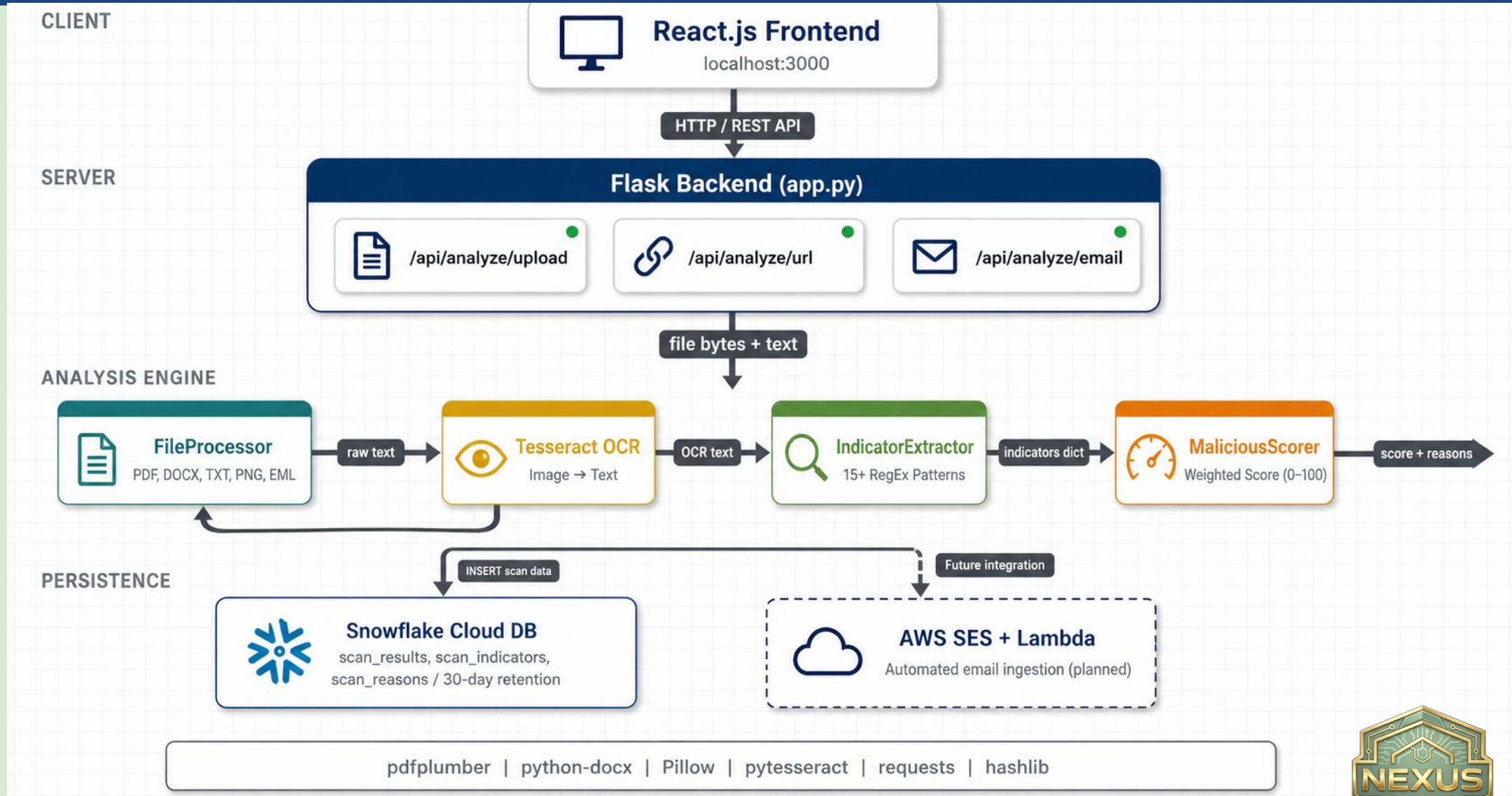


PYTHON LIBRARIES

pdfplumber, python-docx,
Pillow

File format parsing

Tool Integration Diagram



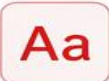
Teaching Maliciousness Through Indicators

NEXUS doesn't just flag threats — it explains why each indicator matters.

HOW NEXUS TEACHES

- **1. Detect**
NEXUS scans the file for suspicious content and patterns.
- **2. Understand**
Each indicator is analyzed in context using threat intelligence and heuristics.
- **3. Explain**
You get clear, human-friendly explanations about why the indicator could be malicious.
- **4. Educate & Protect**
Users learn as they review, building security awareness while staying protected.

EXAMPLE INDICATORS & WHY THEY MATTER

Indicator Found		Why It Could Be Malicious	Risk Impact	
	Keyword: “verify account” Found in document text	Attackers use urgency and account verification themes to trick users into taking action or revealing credentials.	High	
	URL Detected http://secure-login-update.com	The domain is not associated with the expected organization and may lead to a phishing or malicious site.	Medium	
	Embedded File invoice_update.exe	Executable files can run scripts or install malware when opened.	High	
	OCR Text Detected Hidden text in image	Malicious actors hide instructions or links in images to evade basic text-based detection.	Medium	
	Suspicious Pattern Unusual formatting + urgency	Abnormal formatting combined with pressure phrases is a common social engineering technique.	Low	



**We don't just detect.
We help you understand.**



By connecting every finding to a clear explanation, NEXUS turns every scan into a learning opportunity and a stronger defense.

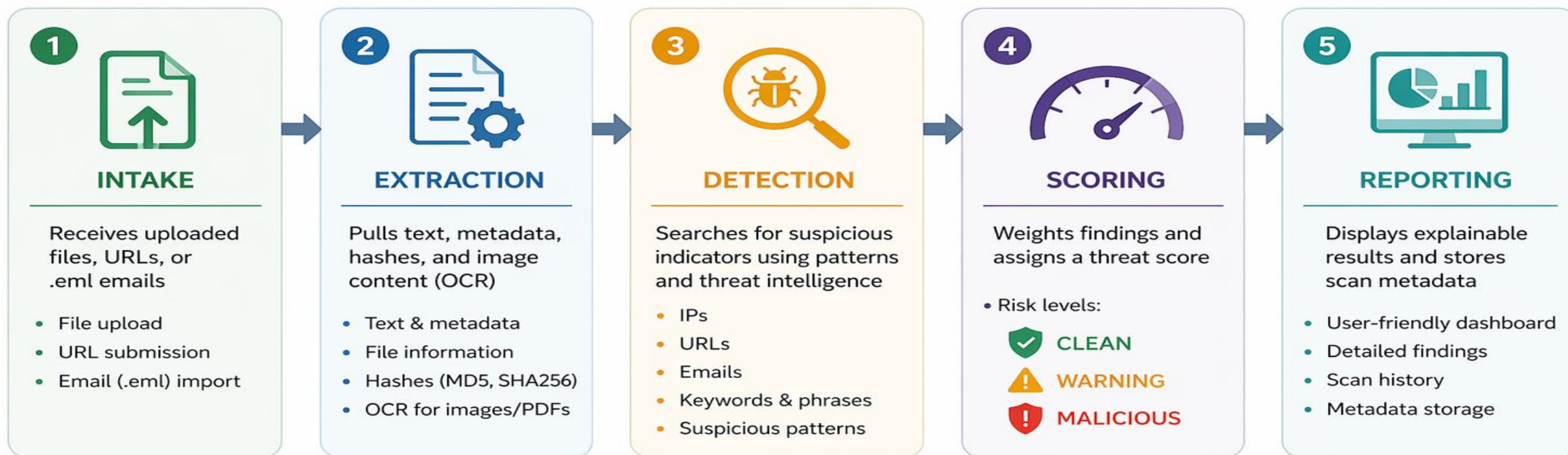


**Knowledge today.
Safer tomorrow.**



BACKEND: How NEXUS Makes Decisions

Our backend pipeline analyzes suspicious inputs and turns them into fast, explainable security decisions.



GOAL

Turn suspicious files into fast, understandable security decisions.



Accurate Analysis



Privacy First



Fast Results



Explainable Outcomes

5. BACKEND: CODE DEEP-DIVE



Backend Architecture & Implementation

POST /api/analyze/upload



- Receives file via multipart form data
- Saves to /tmp with UUID prefix, processes, deletes
- Returns: score, indicators, reasons, hashes

FILE

POST /api/analyze/url



- Receives URL in JSON body
- Downloads in sandbox with 10MB limit, 30s timeout
- Returns: score + URL-specific indicators

URL

POST /api/analyze/email



- Receives .eml file or JSON email payload
- Parses headers, body text, scans attachments
- Returns: combined email + attachment results

EMAIL



All endpoints follow the same 6-step pipeline:
Receive → Process → Extract → Score → Store → Return



main malicious-file-analyzer / backend / app.py

code Blame

```
91 # =====
92 # FILE UPLOAD ANALYSIS ENDPOINT
93 # =====
94
95
96 @app.route("/api/analyze/upload", methods=["POST"])
97 def analyze_upload():
98     """
99     Analyze a file uploaded directly by user.
100
101     Process:
102     1. Receive file from user
103     2. Process file (extract text)
104     3. Extract indicators (URLs, IPs, etc.)
105     4. Calculate malicious score
106     5. Store results in Snowflake
107     6. Return analysis results
108
109     Returns:
110         JSON response with analysis results
111     """
```

```
malicious-file-analyzer / backend / services / file_processor.py

ode Blame

185     def extract_metadata_from_image(self, file_path: str) -> Dict:
198         logger.error("Pillow not available")
199         return {}
200
201         metadata = {}
202         try:
203             image = Image.open(file_path)
204             exif_data = image.getexif()
205
206             if exif_data:
207                 for tag_id, value in exif_data.items():
208                     tag = TAGS.get(tag_id, tag_id)
209                     metadata[str(tag)] = str(value)
210
211             logger.info(f"Extracted {len(metadata)} metadata fields from image")
212         except Exception as e:
213             logger.error(f"Error extracting image metadata: {e}")
214
215         return metadata
216
217     def extract_text_from_image(self, file_path: str) -> str:
218         """
219         Extract text from image using OCR
220
221         WHY: Images may contain phishing URLs, emails, or malicious instructions
222         """
223         if not IMAGE_AVAILABLE or not OCR_AVAILABLE:
224             logger.warning("OCR dependencies not available")
225             return ""
226
227         try:
228             image = Image.open(file_path).convert("L") # grayscale
```

- Validates file exists, size $\leq$ 50MB, supported type
- Calculates MD5 + SHA256 hashes for every file
- Routes to format-specific extractor based on extension
- Returns unified dict: extracted_text, metadata, hashes, file_type
- Supported: PDF, DOCX, PNG, JPG, JPEG, WebP, TXT, EML

Tesseract OCR: Finding Hidden Text

- 4-step image preprocessing before OCRGrayscale → Auto-contrast → Sharpen → Binary threshold (160)
- Catches text hidden in screenshots, scanned docs, steganographic content
- Reliability: 30–60% on images (depends on quality, font, contrast)

```
main malicious-file-analyzer / backend / services / file_processor.py

Code Blame

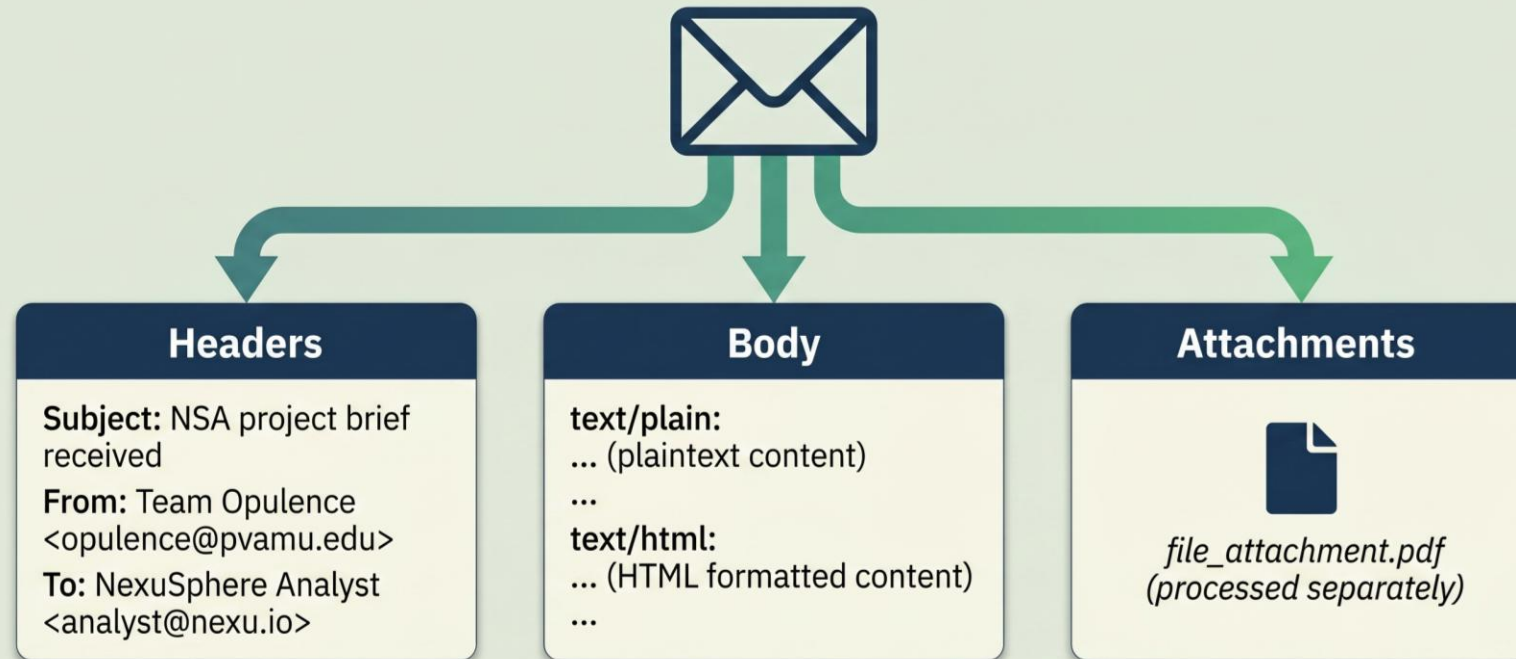
52 class FileProcessor:
150         f"Extracted {len(text)} characters from PDF ({len(pdf.pages)} pages)"
151     )
152     except Exception as e:
153         logger.error(f"Error extracting PDF text: {e}")
154
155     return text
156
157 ✓ def extract_text_from_docx(self, file_path: str) -> str:
158     """
159     Extract text from Word document
160
161     WHY: Word docs are common phishing vehicles
162
163     Args:
164         file_path (str): Path to DOCX file
165
166     Returns:
167         str: Extracted text (all paragraphs)
168     """
169     if not DOCX_AVAILABLE:
170         logger.error("python-docx not available")
171         return ""
172
173     text = ""
174     try:
175         doc = Document(file_path)
176         for paragraph in doc.paragraphs:
177             text += paragraph.text + "\n"
```



Email Analysis: Parsing .eml Files



This diagram shows the first stage of the analysis pipeline. Subsequent steps extract indicators, score results, and store the unified data in Snowflake.



- Uses Python's email.parser.BytesParser with policy.default
- Handles both single-part and multipart emails
- Skips attachment content dispositions (processed separately)
- Currently finalizing full attachment scanning integration

main / malicious-file-analyzer / backend / services / file_processor.py

Code Blame

```
177         text += paragraph.text + "\n"
178
179         logger.info(f"Extracted {len(text)} characters from Word doc")
180     except Exception as e:
181         logger.error(f"Error extracting Word text: {e}")
182
183     return text
184
185 def extract_metadata_from_image(self, file_path: str) -> Dict:
186     """
187     Extract EXIF metadata from image
188
189     WHY: Images can contain GPS coordinates, camera info, timestamps
190
191     Args:
192         file_path (str): Path to image file
193
194     Returns:
195         dict: EXIF metadata key-value pairs
196     """
197     if not IMAGE_AVAILABLE:
198         logger.error("Pillow not available")
199         return {}
200
201     metadata = {}
202     try:
203         image = Image.open(file_path)
204         exif_data = image.getexif()
205
206         if exif_data:
207             for tag_id, value in exif_data.items():
208                 tag = TAGS.get(tag_id, tag_id)
209                 metadata[str(tag)] = str(value)
210
211         logger.info(f"Extracted {len(metadata)} metadata fields from image")
212     except Exception as e:
213         logger.error(f"Error extracting image metadata: {e}")
214
215     return metadata
216
217 def extract_text_from_image(self, file_path: str) -> str:
218     """
219     Extract text from image using OCR
220
221     WHY: Images may contain phishing URLs, emails, or malicious instructions
222     """
223     if not IMAGE_AVAILABLE or not OCR_AVAILABLE:
224         logger.warning("OCR dependencies not available")
225         return ""
```



6. FRONTEND DESIGN & USER EXPERIENCE



User Experience: Designed For Everyone

Simplicity

- 3-click maximum to results
- Minimal cognitive load
- No cybersecurity expertise required

Clarity

- Visual threat scoring
- Color-coded severity (Green/Amber/Red)
- Plain language explanations

Accessibility

- WCAG 2.1 AA compliance
- Screen reader compatible
- Keyboard navigation support
- High contrast ratios

Speed

- Real-time progress indicators
- Instant feedback
- Non-blocking operations

Our Design Approach

"Enterprise-grade security should feel consumer-simple."



Frontend: Technical Design

Technology

- React.js
- Responsive CSS
 - RESTful API
- WebSocket Updates

Features

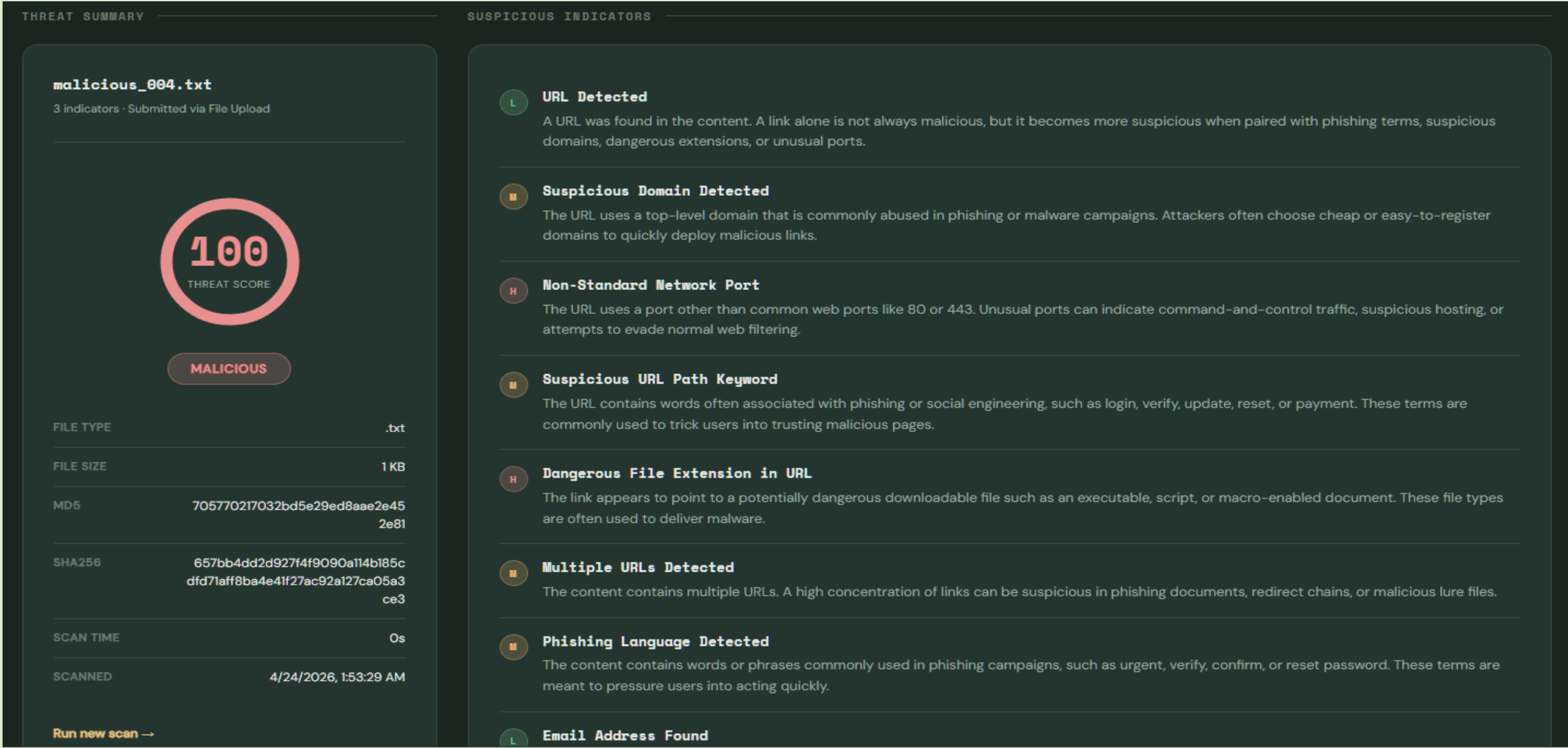
- Drag & Drop Upload
- Real-Time Progress
- Threat Visualization
 - Export Reports

Accessibility

- WCAG 2.1 AA
- Screen Readers
- Keyboard Nav
- Dark/Light Mode

Built for speed, accessibility, and ease of use.

Live System Output: Explainable Threat Results



7. TESTING & PERFORMANCE RESULTS



TESTING DATASET OVERVIEW

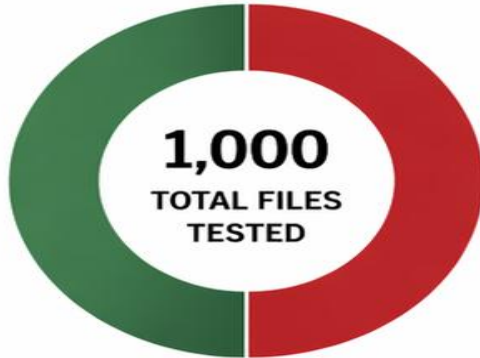
We evaluated NEXUS using a balanced and diverse dataset to ensure reliable and robust results.

DATASET COMPOSITION

500

CLEAN FILES

Legitimate, safe files with no malicious content



500

MALICIOUS FILES

Malware samples and files containing known threat indicators



FILE TYPE DISTRIBUTION



.DOCX

250 files
(25%)



.PDF

250 files
(25%)



.TXT

250 files
(25%)



.PNG

250 files
(25%)



WHAT THIS ENSURES



Balanced Evaluation

Equal mix of clean and malicious files to measure accuracy and reliability.



Format Diversity

Multiple file types to test real-world scenarios and content variations.



Robust Validation

Ensures NEXUS can detect threats across different file formats and content.



Reliable Results

Large dataset strengthens confidence in system performance.



Total: 1,000 Files Tested

500 Clean

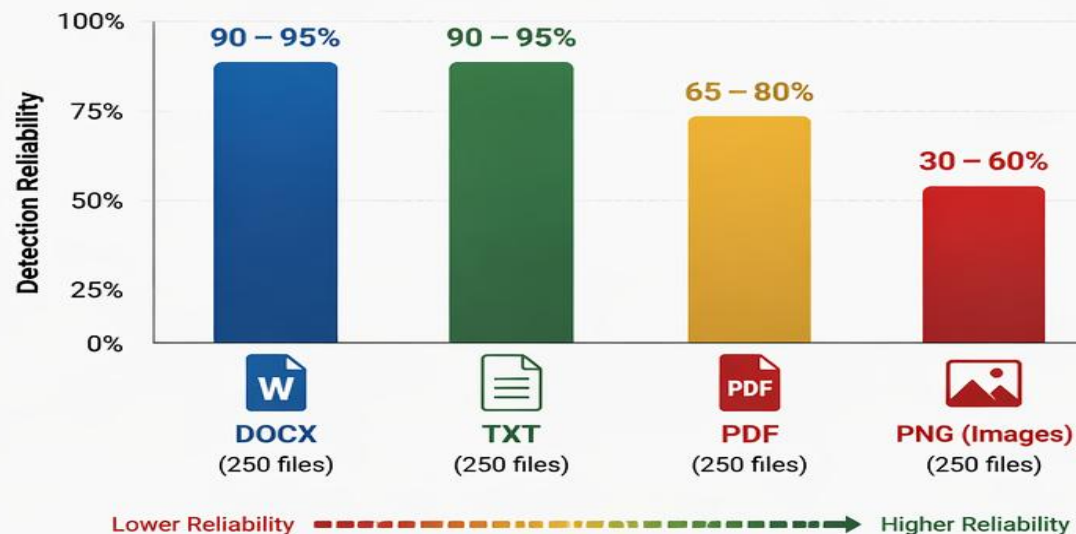
500 Malicious

4 File Types

TESTING RESULTS BY FILE TYPE

NEXUS was evaluated on a dataset of 1,000 files (500 clean, 500 malicious) across 4 file types to measure detection performance and reliability.

DETECTION RELIABILITY BY FILE TYPE



PERFORMANCE SUMMARY BY FILE TYPE

File Type	Detection Reliability	Reason
 DOCX	90 – 95% High	Text is directly readable. Consistent pattern matching and indicator detection.
 TXT	90 – 95% High	Plain text format makes extraction and analysis very reliable.
 PDF	65 – 80% Moderate – High	Performance depends on how text is embedded or scanned within the PDF.
 PNG (Images)	30 – 60% Variable	OCR accuracy depends on image quality, font, contrast, and layout complexity.

KEY TAKEAWAYS



Stronger on Text-Based Files
DOCX and TXT files show the highest reliability due to direct and accurate text extraction.



PDFs Are Moderately Reliable
Detection depends on structure and whether the PDF contains selectable text or scanned content.



Images Are the Biggest Challenge
OCR limitations lead to lower and more variable performance on image-based files such as PNG.



These results highlight the strengths of NEXUS on text-based content and the areas where OCR improvements and advanced preprocessing can further enhance detection in image and scanned documents.

BACKEND THREAT SCORE LOGIC

NEXUS uses a rule-based scoring engine that assigns weighted points to suspicious indicators and additional points for high-risk combinations. The final score is normalized to a range of 0–100.

1. INDIVIDUAL INDICATOR WEIGHTS	
INDICATOR	WEIGHT (POINTS)
 URL Detected	+5
 Multiple URLs (2 or more)	+10
 Email Address Found	+5
 Phishing Language / Keywords (e.g., urgent, verify, account, login)	+15
 Suspicious Domain (low reputation / blacklisted)	+20
 Dangerous File Extension in URL (e.g., .exe, .zip, .scr, .js)	+20
 Non-Standard or Suspicious Port (e.g., :8080, :4444, :8888)	+15

2. COMBINATION BONUSES	
COMBINED INDICATORS	BONUS (POINTS)
 +  URL + Email (appears together)	+15
 +  URL + Phishing Language (appears together)	+20
 +  +  Multiple Indicator Types (3 or more different types)	+25



SCORING NOTES

- Points from all triggered indicators are added together.
- Combination bonuses are applied on top of individual weights.
- Final score is capped at 100 points.

3. FINAL SCORE INTERPRETATION (0–100)		
 0 – 30 CLEAN / LOW RISK No significant threats detected.	 31 – 69 WARNING / MODERATE RISK Suspicious content detected. Exercise caution.	 70 – 100 MALICIOUS / HIGH RISK Highly suspicious content. Immediate attention recommended.



This scoring model is transparent, explainable, and easy to tune as new threat intelligence and patterns are discovered.

8. LIVE DEMONSTRATION



Analysis Method	What We'll Demonstrate
File Upload (Primary Demo)	•Upload malicious PDF/TXT sample •Upload PNG image Sample •Threat score + Indicator Explanations
URL Analysis (Brief Demo)	•Submit Sample URL •Indicator Review •Risk scoring workflow
Email File (.eml) Analysis (Brief Demo)	•Upload .eml sample •Sender/body extraction •Phishing indicator detection
Results Dashboard	•Threat score explanation •Show why item was flagged •User-friendly findings display

9. CONCLUSION



Conclusion & Thank You



NEXUS: Explainable Threat Analysis

- Detects suspicious indicators in files, URLs, and email samples
 - Uses OCR for image-based content
- Provides threat scores with clear explanations
 - Helps users make faster, smarter security decisions

In conclusion, NEXUS demonstrates how accessible, explainable security tools can help users identify suspicious content more efficiently.



Thank you so much for you time!

Any Questions?

